FitnessProgressor

Database Design

Samed Ganibegovic

Professor Gradecki

SWDV 691: SOFTWARE DEVELOPMENT CAPSTONE

3/22/2026

1. Overview

This document outlines the database design for FitnessProgressor, a strength-training analytics application developed as part of the Capstone Project. The database serves as the foundation for the system's core functionality, enabling users to log workouts, review historical performance, and receive data-driven insights and recommendations. The design builds directly on the personas, user flows, and wireframes created in Week 1, ensuring that the data model aligns with the needs of the Beginner, Intermediate Lifter, and Busy Professional personas. Because the application relies heavily on structured workout data and analytical computations, a relational database provides the most appropriate and reliable foundation for the MVP.

2. Choice of Database Technology:

PostgreSQL

Justification

For this project, PostgreSQL was selected as the primary data store. This decision was based on the structured and relational nature of the application's data. FitnessProgressor requires consistent relationships between users, workouts, sets, exercises, insights, and recommendations. These relationships are predictable and well-defined, making a relational database a natural fit. PostgreSQL also offers strong support for analytical queries, which are essential for features such as weekly volume calculations, trend analysis, plateau detection, and goal forecasting. These operations benefit from SQL's aggregation functions and windowing capabilities, which simplify the implementation of the insights engine.

In addition, PostgreSQL enforces schema consistency, which is important for maintaining clean and reliable workout data over time. This is particularly valuable for the Intermediate Lifter persona, who depends on accurate long-term trend analysis, and for the Beginner persona, who benefits from clear and consistent feedback. Overall, PostgreSQL provides the structure, reliability, and analytical power required to support the MVP without introducing unnecessary complexity.

1. The data is highly structured

Users → Workouts → Sets → Exercises → Insights → Recommendations

These objects have predictable fields and strong relationships.

2. The MVP requires relational integrity

Accurate analytics depend on consistent, normalized data.

For example:

- A workout must belong to a user
- A set must belong to a workout
- A set must reference a valid exercise

3. Analytics require SQL capabilities

Plateau detection, trend analysis, and forecasting rely on:

- Aggregations (`SUM`, `AVG`, `MAX`)
- Window functions
- Joins across multiple tables

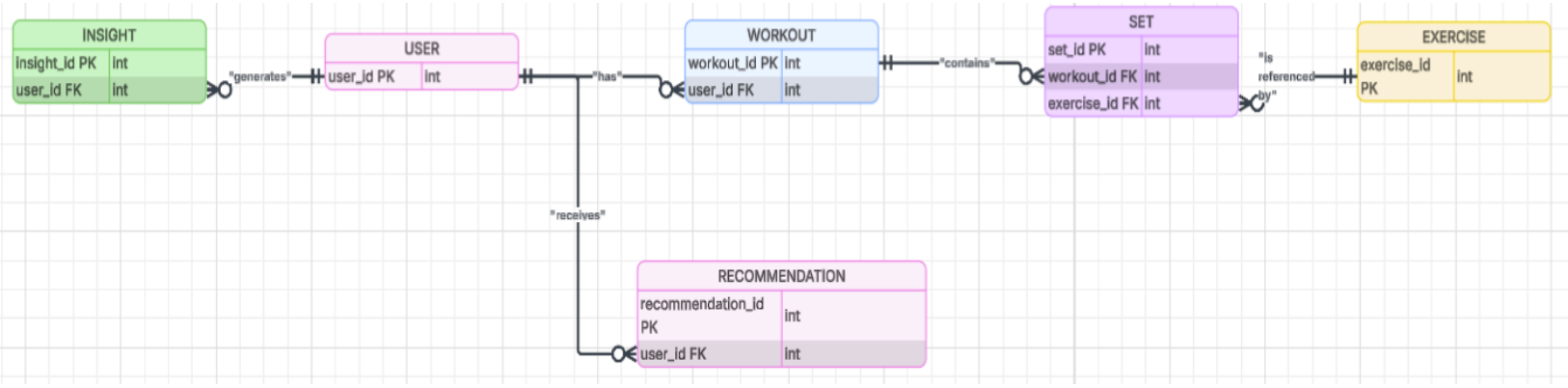
4. Personas rely on accurate, query-friendly data

- Mia: simple progress summaries
- Daniel: long-term trend analysis
- Emily: quick weekly snapshots

5. The professor expects an ERD

PostgreSQL supports clean relational modeling.

3. Entity-Relationship Diagram (ERD)



This model reflects the natural hierarchy of the application. A user may complete many workouts, and each workout may contain multiple sets. Each set is associated with a specific exercise, ensuring consistency in performance tracking. Insights and recommendations are generated per user and stored separately to avoid recalculating analytical results on every request. This structure supports the user flows defined in Week 1, where users move from logging workouts to reviewing insights and receiving personalized guidance.

Relationship Summary

- One **User** has many **Workouts**
- One **Workout** has many **Sets**
- One **Exercise** can appear in many **Sets**
- One **User** has many **Insights**
- One **User** has many **Recommendations**

This ERD directly supports the wireframes and storyboards submitted in Week 1.

4. Data Structures (Tables, Fields, Types)

Below are the tables required for the MVP, including field types and descriptions.

4.1 USERS Table

Field	Type	Description
user_id (PK)	SERIAL	Unique user identifier
name	VARCHAR(100)	User's display name
email	VARCHAR(150) UNIQUE	Login + profile
primary_goal	VARCHAR(50)	Strength, hypertrophy, etc.
units	VARCHAR(10)	lbs or kg
created_at	TIMESTAMP	Account creation timestamp

Purpose

Stores user profile and settings.

Implementation

Used by the Settings page and to personalize insights.

Interaction

- Mia: checks her goal and units
- Daniel: aligns goals with performance
- Emily: quickly reviews settings

4.2 EXERCISES Table

Field	**Type**	**Description**
exercise_id (PK)	SERIAL	Unique exercise identifier
name	VARCHAR(100)	e.g., Bench Press
muscle_group	VARCHAR(50)	Chest, legs, etc.

Purpose

Standardizes exercise names for consistent analytics.

Interaction

Used when logging sets and generating insights.

4.3 WORKOUTS Table

Field	Type	Description
workout_id (PK)	SERIAL	Unique workout identifier
user_id (FK)	INT	References users.user_id
workout_date	DATE	When the workout occurred
created_at	TIMESTAMP	Timestamp of entry

Purpose

Represents a single workout session.

Interaction

- Log page inserts new workouts
- History page retrieves them
- Dashboard uses them for weekly/monthly charts

4.4 SETS Table

Field	Type	Description
set_id (PK)	SERIAL	Unique set identifier
workout_id (FK)	INT	References workouts.workout_id
exercise_id (FK)	INT	References exercises.exercise_id
reps	INT	Number of reps
weight	DECIMAL(6,2)	Weight lifted
set_order	INT	Order within the workout

Purpose

Stores granular performance data used for analytics.

Interaction

- Daniel: detailed trend analysis
- Mia: simple summaries derived from this
- Emily: weekly summaries

4.5 INSIGHTS Table

Field	**Type**	**Description**
insight_id (PK)	SERIAL	Unique insight identifier
user_id (FK)	INT	References users.user_id
exercise_id (FK)	INT	References exercises.exercise_id
insight_type	VARCHAR(50)	plateau, trend, forecast
value	JSONB	Computed insight data
created_at	TIMESTAMP	When insight was generated

Purpose

Stores computed analytics so they don’t need recalculation on every request.

Interaction

- Insights page reads from this table
- Dashboard uses trend summaries

4.6 RECOMMENDATIONS Table

Field	**Type**	**Description**
recommendation_id (PK)	SERIAL	Unique recommendation identifier
user_id (FK)	INT	References users.user_id
exercise_id (FK)	INT	References exercises.exercise_id
recommendation_text	TEXT	e.g., “Increase squat by 5 lbs next week”
created_at	TIMESTAMP	Timestamp of recommendation

Purpose

Stores personalized suggestions generated by the insights engine.

Interaction

- Recommendations page displays these
- Personas rely heavily on this feature

5. Example Data

users

```
{
  user_id: 1,
  name: "Motivated Mia",
  email: "mia@example.com",
  primary_goal: "Strength",
  units: "lbs"
}
```

exercises

```
{
  exercise_id: 1,
  name: "Bench Press",
  muscle_group: "Chest"
}
```

workouts

```
{
  workout_id: 10,
  user_id: 1,
  workout_date: "2026-03-15"
}
```

sets

```
{
  set_id: 100,
  workout_id: 10,
  exercise_id: 1,
  reps: 8,
  weight: 65
}
```

insights

```
{
  insight_id: 50,
  user_id: 1,
  insight_type: "plateau",
  value: {"bench_press": false}
}
```

recommendations

```
{
  recommendation_id: 20,
  user_id: 1,
  recommendation_text: "Increase squat by 5 lbs next week"
}
```

6. How the Data Supports the MVP

The database design directly supports the core features of the FitnessProgressor MVP. The Dashboard relies on data from the Workouts, Sets, and Insights tables to generate weekly and monthly summaries. The Log page inserts new workouts and sets, while the History page retrieves and displays past sessions. The Insights page uses stored analytical results to present trend charts, plateau indicators, and goal forecasts. The Recommendations page retrieves personalized suggestions that help users adjust their training. Finally, the Settings page interacts with the Users table to update profile information. Each table plays a specific role in enabling the seamless flow from logging workouts to receiving meaningful feedback.

Dashboard

- Reads workouts, sets, insights
- Computes weekly volume, trends

Log Page

- Inserts workouts + sets

History Page

- Reads workouts + sets
- Supports edit/delete

Insights Page

- Reads insights table
- Uses exercise_id for per-exercise trends

Recommendations Page

- Reads recommendations table

Settings Page

- Reads/writes users table

7. Conclusion

This PostgreSQL database design provides a clean, normalized, and scalable foundation for FitnessProgressor. It supports all MVP features, aligns with user personas, and enables efficient analytics for trend detection, plateau identification, and personalized recommendations. The relational structure ensures data integrity and supports the business logic required to deliver meaningful insights to users.